Full-Stack Assignment

Extensible Task-Management Platform

Evaluation Focus

- **Server-side (primary):** Clean architecture, proper use of C# and .NET patterns, generic task handling that avoids repetitive conditional logic per task type (think: *how would you add a third task type without touching existing code?*), correct workflow enforcement, and proper separation of concerns.
- **Client-side:** Clear state management, organized and reusable components.

1. Objective

Build a Task-Management Platform that cleanly separates:

- **General workflow rules** (apply to every task, present and future)
- **Task-specific rules** (apply only to a given task type)

We define **Procurement** and **Development** tasks as examples. Your architecture must support additional task types without structural rewrites.

2. Core Workflow Rules

#	Rule
1	A task is assigned to exactly one user at any moment.
2	A task is either Open or Closed. Closed tasks are immutable.
3	Status is tracked by ascending integers: 1, 2, 3...
4	**Forward moves must be sequential (no skipping).**
5	Backward moves are always allowed.
6	**A task may be closed only at its final status.**
7	Every status change must: (a) satisfy type-specific data requirements, (b) record the next assigned user.

> *These rules apply to all current and future task types.*

3. Task Types

3.1 Procurement Task

Status	Meaning	Required Data
1	Created	—
2	Supplier offers received	2 price-quote strings
3	Purchase completed	Receipt string
Closed	—	Only from status 3

3.2 Development Task

Status	Meaning	Required Data
1	Created	—
2	Specification completed	Specification text
3	Development completed	Branch name
4	Distribution completed	Version number
Closed	—	Only from status 4

4. Required Operations

Operation	Purpose
Create Task	Accept task type + initial assigned user (from Users table)
Change Status	Forward/backward with validations; assign next user; save custom fields
Close Task	Only from final status
Get User Tasks	Return tasks assigned to a user

> *No need to manage users — seed a few in the database.*

5. Technical Requirements

Server (.NET)

- ASP.NET Core Web API
- ORM or data access layer of your choice with a SQL database
- C#
- REST API
- Consider how custom fields for different task types will be stored — this is a design decision worth thinking about

Client (React)

- Minimal UI (functionality over styling)
- Create task, manage lifecycle (advance/reverse/close), view user's tasks
- Hard-coded user ID is acceptable

6. Deliverables

- Source code (backend + frontend)
- README with setup instructions (how to run both server and client)
- Migrations with seeded demo users
- A brief explanation (in README or code comments) of your approach to extensibility - how would a new task type be added?

Feel free to ask if anything is unclear.

Good Luck!